

AI Privacy Firewall: A Trusted Privacy Boundary for Agentic Browsing

Agentic Browsing

[Team Name]

[Department / Institute Name]

Smart India Hackathon 2026 — Problem Statement SIH26171

Email: [team-email@example.com]

Abstract— Browser-based AI agents increasingly rely on visual and structural page context to act on a user's behalf. Forwarding that context to a remote model without modification risks exposing sensitive values such as passwords, personal information, and API keys, while the limited compute budget of client devices makes fully local reasoning impractical. This paper presents AI Privacy Firewall, a proposed solution to Smart India Hackathon problem statement SIH26171, issued by the Indian Space Research Organisation (ISRO). The design places a local perception, detection, and sanitization layer between the browser and any remote reasoning model, so that only a sanitized representation of on-screen context ever leaves the device, while a local credential vault keeps secrets out of the remote reasoning path entirely. Remote reasoning and local execution are kept as distinct, clearly bounded stages.

Index Terms— on-device AI, browser agents, privacy-preserving computation, PII detection, sanitization, agentic browsing, human-computer trust boundary.

I. INTRODUCTION

Browser agents are increasingly able to read a web page's visual and structural state and act on it — filling forms, navigating menus, and completing multi-step tasks on a user's behalf. To do this reliably, an agent typically needs some representation of what is on screen: the rendered layout, the Document Object Model (DOM), or a screenshot.

As such agents move from experimental tools to everyday assistants, the context they consume grows to include whatever is rendered on the page at request time. Ordinary web sessions routinely contain login forms, email addresses, phone numbers, names, uploaded documents, and API keys pasted into developer tools. When this context is forwarded, unmodified, to a remote model for reasoning, the sensitive values travel with it.

Client devices generally lack the compute budget to run large-scale reasoning models locally, so heavy reasoning is expected to remain remote. The open design question is therefore how much of the browser's context, and in what form, should ever leave the device in the first place. This paper proposes AI Privacy Firewall as an answer to that question for SIH26171 [1].

II. Problem Statement and Analysis

The official SIH26171 problem statement calls for a hybrid architecture in which lightweight visual perception runs on the client device while heavier reasoning may run remotely, with sensitive information sanitized locally before any network transmission. The official evaluation weighs five criteria: accuracy of visual context extraction (25%), precision and recall of sensitive-data detection (20%), precision of redaction (20%), client-side resource utilization (20%), and end-to-end latency (15%).

Two requirements are in tension. A browser agent needs enough context to act correctly — it must know that a field is a login form, that a button submits an order, or that a table row contains a shipping address. At the same time, the values held in those fields are frequently private. A naive integration forwards raw screenshots or raw DOM dumps to a remote model on every request; this satisfies the capability requirement but not the privacy requirement, since any sensitive value present on screen is exposed regardless of whether the current task needs it. The practical design space is therefore a split architecture: lightweight local processing responsible for perception and privacy, and a remote model responsible for reasoning over whatever the local layer determines is safe to share.

III. Proposed System

AI Privacy Firewall inserts a local enforcement layer between the browser and any remote reasoning model. The design is summarized by a four-stage principle, each bound to a location of execution: see locally, protect locally, reason remotely, act locally. The local device perceives the page using DOM analysis and a lightweight vision model, detects and sanitizes sensitive values, and gates what is allowed to leave the device. Only the sanitized representation is sent to a remote vision-language model (VLM) or large language model (LLM) for reasoning. The remote model returns a structured action, which a local executor carries out in the real browser; wherever a locally held credential is required, it is supplied from a local vault and never transmitted to the remote model.

III-A. System Architecture

Fig. 1 shows the end-to-end data path. Local components execute entirely on the user's device; the remote reasoning component is the only stage outside the local trust boundary.

User → Browser (Chrome / Firefox) → Local Perception (DOM Analysis + Vision AI) → AI Privacy Firewall (Detect → Sanitize → Gate) → Remote VLM / LLM (Reasoning) → Local Browser Executor → Browser (Chrome / Firefox). A parallel Local Privacy Vault supplies credentials directly to the executor.

III-B. Local Visual Perception

The local perception layer combines two complementary signals: structured DOM analysis and a lightweight on-device vision model. DOM analysis is precise for standard HTML form elements but blind to content rendered in a canvas element or embedded inside an image; vision analysis covers those visual cases but is, on its own, more error-prone at inferring the semantic role of a structured input field. Combining both signals allows the firewall to cross-check detections and reduce both categories of error.

III-C. DOM Analysis

DOM analysis inspects page structure and semantic HTML attributes to identify fields likely to hold sensitive data — for example, input types, form structure, and labelled fields. Given the elements , , and Login, the firewall infers that the email and password fields are sensitive while the login button is an ordinary, non-sensitive UI element that requires no redaction.

III-D. Privacy Detection

Privacy detection consists of PII detection (names, email addresses, phone numbers), sensitive-field detection (password and credential inputs), secret and API-key detection in page or script context, and visual sensitive-region detection (faces and other PII rendered as pixels rather than text). Detection results are combined into a single per-element sensitivity assessment that downstream sanitization consumes.

III-E. Sanitization Engine

Sanitization is defined as transforming browser or visual context locally so that sensitive information becomes unidentifiable before transmission, while preserving enough semantic information for the requesting agent to complete its task — that is, preserving semantics while removing sensitive values. As an illustration using synthetic data, the fields Email: user@example.com and Password: ***** are transmitted to the remote model as Email: [EMAIL] and Password: [PASSWORD], while the Login button is left unmodified. The user's actual browser is not altered; sanitization applies only to the copy of the context that would otherwise be transmitted remotely. Not every category of sensitive information is handled identically — a detected face may be redacted visually, while a text field is replaced with a typed placeholder token such as [EMAIL], [PHONE], [NAME], or [SECRET].

III-F. Outbound Transmission Gate

Every context request bound for the remote model passes through an outbound transmission gate, the single enforcement point for what may leave the device. If a value cannot be safely sanitized — for instance, because redacting it would destroy information the task genuinely requires — the gate may block the transmission, request explicit user approval, or apply a predefined privacy policy. The remote model is not the authority that decides whether private information is allowed to leave the device; the local firewall is. As a supporting capability, the firewall also inspects relevant page and script context for accidentally exposed secrets, such as an API key assigned to a variable in an inline script, and treats a match the same way as any other sensitive value. This capability is scoped narrowly in support of the privacy-preserving-browsing goal and does not extend the system into a general antivirus or vulnerability-scanning product.

III-G. Local Privacy Vault

The local privacy vault, inspired by browser password managers, stores credentials on-device and fills authorized fields locally, so that the remote model never needs to receive the underlying secret to complete a login or similar workflow. For prototype and demonstration purposes only synthetic credentials are used; production-grade secure credential storage is not claimed unless it has been implemented and independently verified.

III-H. Remote Reasoning Agent

The remote VLM/LLM performs the reasoning step that current client hardware cannot economically perform. It receives only the sanitized representation of the page and returns a structured action rather than free-form output — for example, given a sanitized login page description, the model returns an action such as {"action": "CLICK", "target": "Login"}. This maintains a clean separation of responsibility: reasoning is remote, while privacy enforcement and execution remain local.

III-I. Local Browser Executor

The local executor receives the structured action returned by the remote model and carries it out against the user's actual browser session, drawing on local credentials where required and never re-exposing them to the remote model.

IV. User Interface and Privacy Dashboard

The browser surfaces a persistent protection indicator — for example, "AI Privacy Firewall: ON" — together with an ON/OFF control. A visible boundary or badge communicates only that protection is currently active; it is a status indicator, not itself a security mechanism. A privacy dashboard, modelled on the privacy-report concept used by privacy-focused browsers, lists what has been

detected, sanitized, or blocked, and summarizes recent AI requests as safe, sanitized, or blocked. Any counts shown in dashboard mock-ups are illustrative UI examples, not measured experimental results.

V. END-TO-END WORKFLOW AND PROTOTYPE DEMONSTRATION

A complete interaction proceeds as follows: the user enables the firewall; local DOM and visual perception inspect the current page state; sensitive elements are detected and sanitized locally; the sanitized context is transmitted to the remote model, which returns a structured action; the local executor performs the action, drawing on the local vault where a credential is required; and the privacy dashboard records what was protected, sanitized, or blocked.

A short prototype demonstration, using only synthetic data, exercises this workflow end to end. A demo web page is populated with a fake name, email address, phone number, password, and API key, alongside ordinary UI elements. With the firewall enabled, the demonstration shows the original context, the sanitized context, confirmation that only sanitized context reaches the AI or a mock server, the returned action, its execution in the local browser, and the resulting privacy-dashboard entry. This directly demonstrates the central SIH26171 requirement: sensitive browser context is sanitized locally before any information reaches a remote model.

VI. Implementation and Technology Stack

The following stack is proposed for implementation; components are described as implemented only where explicitly stated. On the client, the system targets a Chrome/Firefox extension built on Manifest V3 [2], using JavaScript/TypeScript and DOM APIs, with browser-native acceleration via WebGPU [3] and WebAssembly [4]. Local inference is proposed via a lightweight Vision Transformer or equivalent model served through ONNX Runtime Web [5] and/or Transformers.js [6]. The server side consists of a remote VLM/LLM for reasoning, exposed through a Python/FastAPI (or equivalent) backend. The privacy layer spans DOM analysis, PII detection, visual detection, secret detection, sanitization, and the outbound transmission gate described in Section III-F.

VII. Evaluation Methodology

The official SIH26171 evaluation weighs five criteria. For each, we state what is measured, how it will be tested, and how the proposed architecture is intended to address it. No numerical results are claimed in this paper; entries are marked TBD (to be determined) where a measured value would otherwise be required.

VIII. Security and Threat Model

We assume the user, the local browser, the local privacy firewall, and the local privacy vault are trusted; the remote AI/VLM/LLM is treated as untrusted and external. The guiding principle is minimum necessary context: the remote model should receive only the information required for the current reasoning step. Threats considered include sensitive visual information leaking to the remote model, password or API-key exposure, PII exposure, over-redaction that reduces the remote model's usefulness, under-redaction that causes a privacy leak, malicious or compromised page content, and both false positives and false negatives in sensitivity classification. The outbound transmission gate is the primary mitigation for leakage threats, since it is the single point through which all outbound context must pass; detection errors are mitigated, though not eliminated, by combining DOM and vision signals. No claim of complete or perfect security is made.

IX. Comparison with Conventional Cloud Agents

A conventional agent forwards raw context directly from the browser to a remote model, risking exposure of any sensitive browser context regardless of task relevance. The proposed system instead routes context through local perception and the privacy firewall before transmission, so that only sanitized context reaches the remote model and execution remains local. This comparison is conceptual and makes no claim about the internal design of any specific named commercial product.

X. PROTOTYPE SCOPE VERSUS FULL SYSTEM

To avoid overstating what the hackathon prototype implements, we distinguish prototype (MVP) scope from full-system scope.

Prototype: local DOM detection; rule-based sensitive-data detection where appropriate; sanitization of detected values; privacy dashboard; mock or local AI interaction; demonstration of a resulting browser action.

Full system: a fully integrated lightweight on-device Vision Transformer or equivalent; browser-native hardware acceleration; robust visual PII detection; advanced, semantics-preserving sanitization; full remote VLM/LLM integration; comprehensive browser-action execution; and formal evaluation against the criteria in Section VII.

XI. Innovation and Differentiation

The claimed contribution is the integrated architecture, not any single technique in isolation: establishing a trusted local privacy boundary between browser context and a remote agentic AI. Contributing elements include local-first privacy enforcement, hybrid DOM-plus-visual perception, local secret detection, sanitization prior to transmission, an outbound request gate, local credential handling, a human-readable privacy report, a browser-visible protection status, and the explicit separation of remote reasoning from local execution. Individual techniques such as PII detection or credential vaulting are not claimed to be novel in themselves.

XII. Limitations

Local vision models may have lower accuracy than large cloud models.

PII detection can produce both false positives and false negatives.

DOM inspection cannot detect every visually rendered sensitive element.

Excessive redaction can reduce the remote model's reasoning quality.

Browser extension permissions require careful security design.

A production-grade credential vault requires rigorous, independently reviewed security engineering beyond prototype scope.

No privacy detector can guarantee perfect detection of every sensitive element.

XIII. Future Work

More capable on-device vision models and improved DOM-and-vision fusion.

More accurate semantic redaction and per-website privacy policies.

Enterprise deployment and broader browser support.

Hardware-accelerated local inference and stronger local credential management.

Formal, independent security testing.

Adaptive privacy levels based on context sensitivity.

XIV. Conclusion

An AI agent should be able to understand and operate a browser without automatically gaining access to everything the browser knows. AI Privacy Firewall proposes a local enforcement layer — perception, detection, and sanitization on the device, reasoning remote, and execution local — as one way to meet that goal for the SIH26171 problem statement: see locally, protect locally, reason remotely, act locally.

VII-A. Evaluation Criteria

- Accuracy of visual context (25%): measured against manually annotated test pages; addressed by hybrid DOM-plus-vision perception. TBD.
- Precision/recall of sensitive-data detection (20%): measured against a labelled set of synthetic pages with known sensitive and non-sensitive elements. TBD.
- Precision of redaction (20%): measured by comparing pre- and post-sanitization context on the same test set. TBD.
- Client-side resource utilization (20%): measured by profiling the extension under representative browsing sessions. TBD.
- End-to-end latency (15%): measured by timed trials across the workflow. TBD.

REFERENCES

- [1] Indian Space Research Organisation (ISRO), "On-device Visual Perception for Light-weight Browser Agents," Smart India Hackathon 2026 Problem Statement SIH26171.
- [2] Google Chrome Developers, "Chrome Extensions Manifest V3," Online.
- [3] W3C, "WebGPU," Online.
- [4] W3C, "WebAssembly Core Specification," Online.
- [5] Microsoft, "ONNX Runtime Web," Online.
- [6] Hugging Face, "Transformers.js," Online.